TP Final — Gymbro AI

Informe de los flujos de n8n: WF1 (Alertas), WF2 (Backups), WF-RESTORE (Prueba de restauración) y WF3 (Informe IA)

Estructura y funcionamiento interno, y carpetas creadas en infra/n8n

---

Punto 13 — Automatización de Procesos

1. Panorama general

Tres flujos con responsabilidades distintas y complementarias: **WF1 vigila** (lee Prometheus y crea tickets en MantisBT), **WF2 respalda y verifica** (genera el dump diario y comprueba que es válido) y **WF-RESTORE**(restaura el último dump en un Postgres) y **WF3 resume y avisa** (genera un informe semanal de rendimiento asistido por "IA" y lo distribuye).

La carpeta **infra/n8n** contiene la instancia independiente y segura de **n8n** que orquesta la automatización del TP final de Gymbro AI e integra sus servicios principales: Prometheus, MantisBT, PostgreSQL y el sistema de backups. Incluye:

- **Una imagen Docker propia** (n8nio-gymbro/n8n:2.44.6) que agrega a n8n el cliente PostgreSQL 17.11 (pg_dump, pg_restore y psql) copiando solo los binarios y sonames necesarios desde postgres:17-alpine, porque la imagen oficial de n8n no trae cliente de base de datos.
- **Un compose con 3 servicios**: n8n (UI en http://127.0.0.1:5678), un sidecar PostgreSQL descartable (pg-restore-test, sin puertos publicados y con datos en tmpfs) contenedor secundario de base de datos PostgreSQL que se ejecuta junto al contenedor principal de una aplicación dentro del mismo entorno y que está diseñado para ser **destruido y recreado sin conservar datos permanentes** utilizado para para pruebas de restauración aisladas, y un microservicio Python de stdlib pura (informe-analyzer) que actúa de «motor de análisis» para el informe semanal.
- **Cuatro workflows exportados en JSON**: WF1 (alertas de infraestructura, de Prometheus hacia MantisBT), WF2 (backup diario 03:00 con verificación completa y log en disco), WF3 (informe semanal asistido por IA heurística, con subida a Nextcloud y aviso por email) y WF-RESTORE (prueba controlada de restauración, solo manual).
- **Dos scripts de siembra** (PowerShell y Bash) que dejan una instalación nueva lista con un solo comando: verifican Docker y el .env, generan claves propias por máquina, levantan el stack, crean el owner, importan credenciales y workflows sin duplicar, publican los workflows programados y verifican la conectividad sin imprimir secretos.
- **Artefactos en disco**: logs de backup, de restauración y de informes; un informe generado por WF3; y un dump real de PostgreSQL (72.366 bytes, 131 objetos) creado por WF2 el 2026-11-09.

2. Infraestructura que comparten los flujos

2.1 Contenedores (infra/n8n/compose.yaml)

Contenedor	Función
gymbro-n8n	n8n 2.44.6 con imagen propia (ver 3.2). Puerto 5678. Persiste en volumen n8n_data. Monta ./informes y ./backups en /data (los flujos leen y escriben ahí), y dos env de solo-lectura con credenciales.
gymbro-pgrestore-test	PostgreSQL 17 desechable para la prueba de restauración. Sin puertos publicados, datos en tmpfs, credenciales propias. Nunca toca la base principal.

2.2 Imagen propia (Dockerfile)

La imagen (n8nio-gymbro/n8n:2.44.6) que agrega a n8n el cliente PostgreSQL 17.11 (pg_dump, pg_restore y psql) copiando solo los binarios y sonames necesarios desde postgres:17-alpine, porque la imagen oficial de n8n no trae cliente de base de datos.

2.3 Credenciales y variables (nunca en el código ni en los JSON)

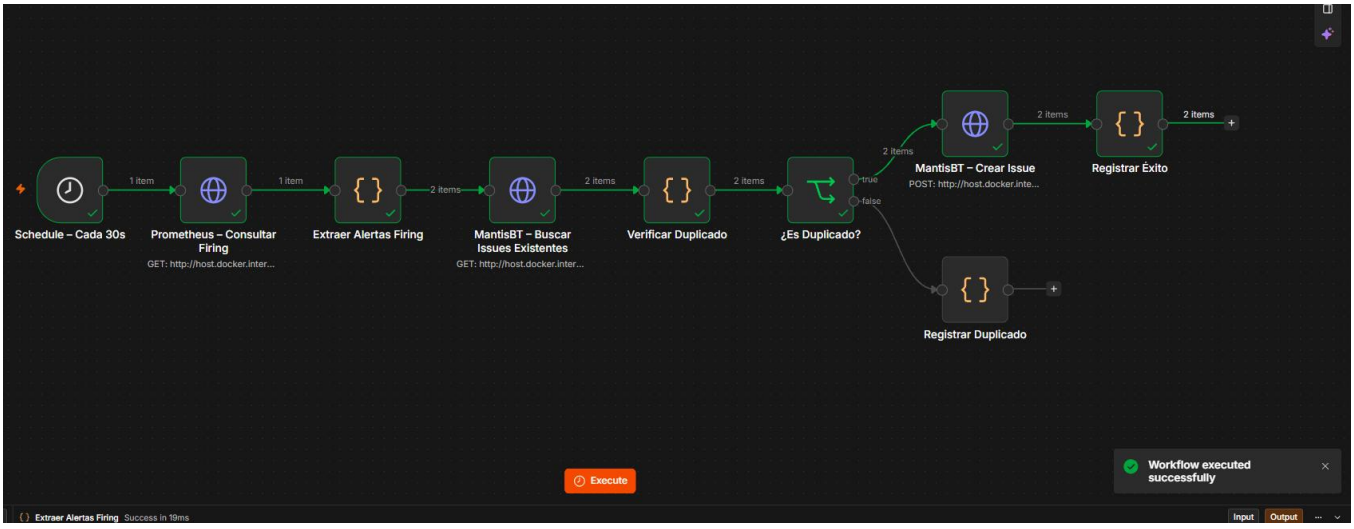
Archivo / credencial	Contenido	Usado por
backup.env (gitignored)	Variables libpq de la BASE PRINCIPAL (PGHOST, PGPORT, PGUSER, PGDATABASE, PGPASSWORD). Montado en /data/backup-config.env	WF2 · nodo Ejecutar pg_dump
restore-test.env (gitignored)	Variables libpq del SIDECAR de prueba (RESTORE_TEST_HOST/PORT/USER + PGPASSWORD). Montado en /data/restore-test-config.env	WF-RESTORE · nodo Restaurar Backup
MantisBT API Token (n8n, httpHeaderAuth)	Token para la API REST de MantisBT	WF1 · nodos MantisBT
Restore Test Scratch (n8n, postgres)	Conexión a la base restore_test del sidecar	WF-RESTORE · nodos Postgres
SMTP account (n8n, smtp)	Cuenta de correo para notificaciones de error (Gmail con clave de aplicación)	WF2, WF-RESTORE y WF3 · nodos de email
Nextcloud WebDAV (sonia) (n8n, httpBasicAuth)	Acceso WebDAV a Nextcloud para subir el informe semanal (carpeta SmartGymBro-TP-Final/05-Monitoreo-y-Automatizacion). Requiere NEXTCLOUD_URL en el entorno	WF3 · nodo Subir a Nextcloud (WebDAV)

3. WF1 — Alertas de Infraestructura

3.1 Qué hace?

Vigilar las reglas de alerta de Prometheus cada 30 segundos y abrir incidencias en MantisBT cuando una alerta está firing, evitando duplicados.

Schedule → Prometheus – Consultar Firing → Extraer Alertas Firing → MantisBT – Buscar Issues Existentes → Verificar Duplicado → ¿Es Duplicado? → (rama false) MantisBT – Crear Issue → Registrar Éxito · (rama true) Registrar Duplicado.



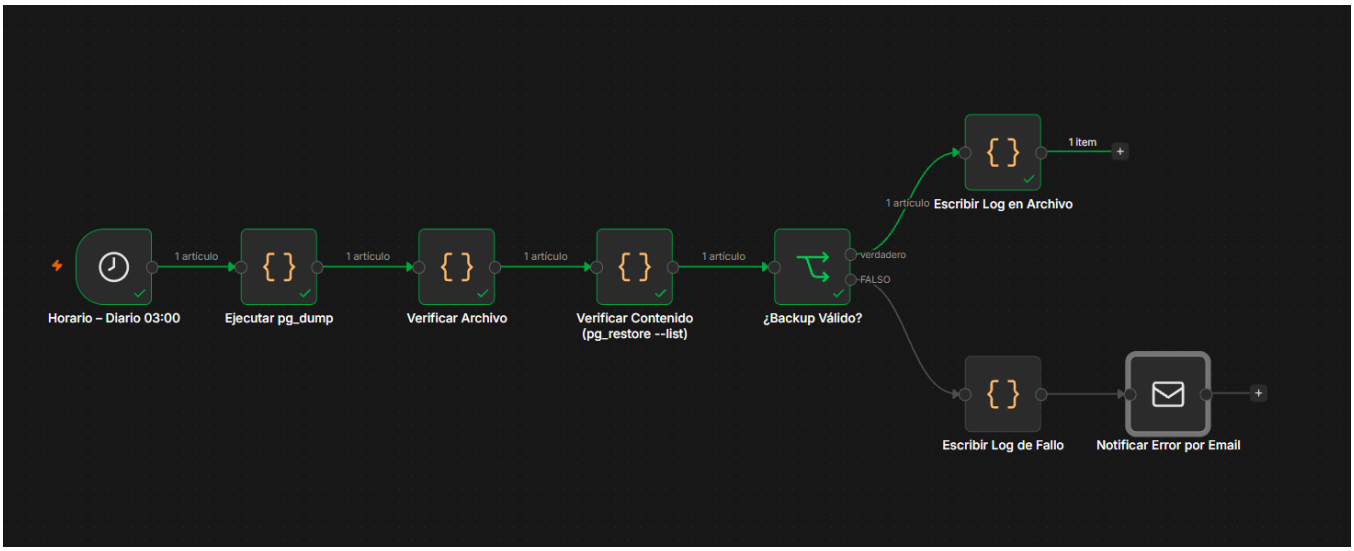
3.2 Flujo de nodos (de izquierda a derecha)

#	Nodo	Qué hace
1	Schedule – Cada 30s	Disparador periódico (intervalSeconds = 30).
2	Prometheus – Consultar Firing	GET http://host.docker.internal:9090/api/v1/rules y espera JSON.
3	Extraer Alertas Firing	Código JS: recorre data.groups[].rules[], se queda con las de type=alerting y state=firing y arma una alerta normalizada (alertname, severity, instance, job, summary, description, startsAt, fingerprint). El fingerprint es la clave de deduplicación: nombre-instancia saneado.
4	MantisBT – Buscar Issues Existentes	GET /api/rest/issues/?page_size=50&sort=id&direction=DESC con el token. Devuelve los últimos 50 tickets.
5	Verificar Duplicado	Código JS: busca en los issues un ticket ABIERTO cuyo summary contenga el fingerprint. Si existe, marca isDuplicate=true; si no, arma summary y description listos para MantisBT (incluye evidencia: instancia, severidad, job, horario de inicio).
6	¿Es Duplicado?	IF booleano: solo pasa a crear ticket si isDuplicate = false.
7	MantisBT – Crear Issue	POST /api/rest/issues/ con summary, description, project id=1, category General, severidad mapeada (critical→critical, warning→major, resto→minor), prioridad (critical→urgente), estado new.
8	Registrar Éxito	Código JS: loguea el issueId creado (resultado=issue_creado) en la salida de n8n.
—	Registrar Duplicado	Rama alternativa: loguea duplicado_ignorado con el fingerprint (no crea ticket).

4. WF2 — Verificación de Backups

4.1 Qué hace en una frase

Respaldar la base smartgymbro_tp a diario y demostrar que el backup es usable: archivo fresco, no vacío y con contenido legible (pg_restore --list). En caso de fallo, registra y notifica por email.



4.2 Flujo de nodos

#	Nodo	Qué hace
1	Schedule – Diario 03:00	Disparador cron 0 3 * * * (hora de Buenos Aires).
2	Ejecutar pg_dump	Arma el nombre gymbro-backup-<ISO>.dump en /data/backups (= infra/n8n/backups/) y corre pg_dump -Fc. Carga las credenciales desde /data/backup-config.env; la password nunca aparece en el JSON ni en los datos de ejecución. Captura exitCode/stderr sin cortar el flujo.
3	Verificar Archivo	Código JS: valida que pg_dump salió con 0, que el archivo existe, pesa > 1 KB y su mtime corresponde a ESTA ejecución (freshness: mtime >= startedAt-5s). Resultado: isValid + error tipificado.
4	Verificar Contenido (pg_restore --list)	Lee el índice del dump (sin tocar la base) y cuenta objetos; exige exitCode 0 y objetos > 0. El propio código advierte: esto NO es una prueba de restauración real.
5	¿Backup Válido?	IF: isValid && contentOk.
6	Escribir Log en Archivo	Rama OK: anexa a backup-log.txt la línea resultado=backup_exitoso | archivo=… | objetos=NNN | error=-
7	Escribir Log de Fallo	Rama FALLO: arma el motivo exacto (pg_dump falló / archivo ilegible / dump vacío), anexa a backup-log.txt una línea resultado=backup_fallido | gen=… | verificacion_archivo=… | contenido=… | error=<motivo>
8	Notificar Error por Email	Rama FALLO: correo desde/hacia gsoniaemiliana@gmail.com, asunto [GYMBRO] Error en backup <archivo>, con fecha, etapa y motivo.

4.3 Qué significa cada comprobación

Comprobación	Por qué importa
pg_dump con código 0	Sin esto el dump puede estar truncado o vacío.
Existe y pesa > 1 KB	Detecta escrituras fallidas o dumps vacíos.
Frescura (mtime ≥ inicio − 5 s)	Evita confundir un dump viejo con uno nuevo ante fallos de escritura.
pg_restore --list con objetos > 0	El dump es legible y tiene contenido. No sustituye a la restauración real.

Nota: WF2 no envía correo cuando el backup sale bien (solo log). El correo de fallo depende del SMTP válido y de que el TLS no sea interceptado (Avast); si el envío falla, el log igualmente queda registrado.

5. WF-RESTORE — Prueba Controlada de Restauración

5.1 Qué hace en una frase

Bajo demanda manual, toma el .dump más reciente y válido, y demuestra que se puede restaurar de punta a punta en un PostgreSQL descartable (el sidecar gymbro-pgrestore-test): verifica tablas, cuenta filas, limpia la arena y deja evidencia en el log; si algo falla, avisa por correo.

Schedule → Ejecutar pg_dump → Verificar Archivo → Verificar Contenido → ¿Backup Válido? → (rama éxito) Escribir Log en Archivo · (rama fallo) Escribir Log de Fallo → Notificar Error por Email.

5.2 Flujo de nodos

#	Nodo	Qué hace
1	Ejecución Manual	Disparador manual (no está programado). Todos los nodos tienen continueRegularOutput para no cortar la cadena ante errores.
2	Seleccionar Backup Válido	Lista /data/backups, filtra gymbro-backup-*.dump > 1 KB, ordena por mtime y elige el MÁS RECIENTE que supere pg_restore --list (exitCode 0 y objetos > 0). Si ninguno sirve, marca selectionOk=false con etapa=seleccionar.
3	Vaciar Scratch	Postgres (credencial Restore Test Scratch): DROP SCHEMA CASCADE de todos los schemas de usuario y recrea public. Lienzo vacío garantizado.
4	Restaurar Backup	pg_restore --exit-on-error --no-owner --no-privileges contra restore_test (credenciales de /data/restore-test-config.env). NUNCA contra la base principal.
5	Contar Tablas	SELECT count(*) de tablas de usuario (debe ser > 0).
6	Verificar Filas (> 0)	Bloque SQL con doble bucle: recorre todas las tablas y suma COUNT(*); exige filas > 0 (el backup real restaura 16 tablas y 199 filas).
7	Limpiar Scratch (Final)	Vuelve a vaciar restore_test: la prueba no deja residuos.
8	Registrar Resultado	Anexa a restore-test-log.txt: resultado=restore_ok|restore_fallido | archivo=… | objetos=… | tablas=… | filas=… | limpieza=OK | etapa=… | error=…
9	¿Restauración OK?	IF: restoreOk = selección OK + restauración OK + tablas > 0 + filas > 0.
10	Notificar Error por Email	Rama FALLO: correo [GYMBRO] Error en prueba de restauración <archivo> con etapa y motivo.

5.3 La arena de pruebas (sidecar)

- **Aislamiento:** Imagen postgres:17-alpine, nombre gymbro-pgrestore-test, red gymbro-n8n-net.
- **Efímero:** tmpfs (los datos viven en memoria y mueren con el contenedor), sin puertos publicados al host.
- **Credenciales propias:** Usuario restore_test / base restore_test; password solo en RESTORE_TEST_PASSWORD (.env local, gitignored).
- **Repetible:** DROP/CREATE SCHEMA antes y después de cada prueba → estado conocido e limpio siempre.

6. WF3 — Informe de Rendimiento Asistido por IA

6.1 Qué hace en una frase

Todos los lunes a las 08:00 recolecta métricas reales de Prometheus, se las pasa a un mini servicio Python analizador que detecta anomalías con reglas de umbral fijas, genera un informe en Markdown, lo guarda en informes/, opcionalmente lo sube a Nextcloud y avisa por correo.

6.2 Flujo de nodos

#	Nodo	Qué hace
1	Schedule – Lunes 08:00	Disparador semanal (triggerAtDay=1, 08:00). Todos los nodos tienen continueRegularOutput.
2	Recolectar Métricas	Instant queries a http://host.docker.internal:9090/api/v1/query (con wget): probe_success y probe_duration_seconds (blackbox_backend), docker_container_restart_count{name="gymbro-backend"}, node_memory_MemAvailable_bytes,

#	Nodo	Qué hace
		node_memory_MemTotal_bytes. Además chequea si existe la variable NEXTCLOUD_URL (define la rama siguiente).
3	Analizar con IA Python	POST http://informe-analyzer:8090/analyze con {"metrics": {...}}. Escribe el payload en /tmp y parsea la respuesta: { anomalies: [...], riskLevel: OK|WARN|CRIT, summary: "..." }. Si el analizador no responde, continúa con riskLevel=UNKNOWN y un resumen de error.
4	Guardar Informe Local (.md)	Renderiza el informe (Generado / Riesgo / Resumen / Anomalías / Métricas observadas / Notas) y lo escribe en /data/informes/informe-<ISO>.md.
5	¿Nextcloud Configurado?	IF booleano: solo sube si había NEXTCLOUD_URL en el entorno.
6	Subir a Nextcloud (WebDAV)	Rama Nextcloud: PUT a NEXTCLOUD_URL/SmartGymBro-TP-Final/05-Monitoreo-y-Automatizacion/<filename>, body raw text/markdown, credencial httpBasicAuth “Nextcloud WebDAV (sonia)”.
7	Notificar por Email	Correo [GYMBRO] Informe semanal – <filename> desde/hacia gsoniaemiliana@gmail.com con riesgo y resumen (lee el nodo Guardar, no el resultado del mail).
8	Registrar en Log de Informes	Anexa a /data/informes/informes-log.txt una línea INFORME: <filename> | riesgo=… | anomalias=N | filepath=… (leyendo del nodo Guardar).

6.3 El servicio informe-analyzer (puerto 8090)

Es un servicio Python stdlib puro (sin pip installs) en informe-analyzer/analyzer.py. Expone:

Endpoint	Método	Uso
/health	GET	Liveness: {"status": "ok"}.
/analyze	POST	Recibe {"metrics": {...}} y devuelve anomalías + riskLevel + summary.

Umbrales heurísticos (ajustables en THRESHOLDS dentro del script):

Métrica	WARN	CRIT
Latencia del probe (backend_probe_duration_seconds)	> 0.5 s	> 2.0 s
Memoria disponible (node_memory_MemAvailable_bytes)	< 512 MB	< 256 MB
Reinicios de contenedor (docker_container_restart_count_total)	> 1	—
Backend caído (backend_up == false)	—	CRIT directo

7. Carpetas creadas en infra/n8n y para qué sirven

Carpeta / archivo	Para qué sirve	¿En git?
workflows/	Exports JSON de los workflows (workflow-1-alertas, workflow-2-backups, workflow-3-informe, workflow-restore-test) + README breve. Son la fuente versionable de los flujos.	Sí

Carpeta / archivo	Para qué sirve	¿En git?
backups/	Dumps .dump generados por WF2 (12 archivos, ~72 KB c/u, 131 objetos). Retención práctica: se van acumulando; conviene rotar/borrar viejos.	No (*.dump gitignored)
informes/	Logs legibles que escriben los flujos: backup-log.txt (éxito/fallo de cada corrida) y restore-test-log.txt (resultado de cada prueba de restauración). PDFs gitignored.	Logs sí; PDFs no
informe-analyzer/	Servicio Python ligero (stdlib puro) para el FUTURO WF3 Informe IA: expone /health y /analyze y devuelve anomalías + riskLevel. Documentado en su README.	Sí
compose.yaml / Dockerfile	Definición del servicio n8n (+ sidecar de restauración) e imagen propia con cliente PostgreSQL.	Sí
.env / .env.example / .gitignore	Entorno local del módulo; .env real gitignored, .env.example con placeholders documentados.	.env no; resto sí
backup.env / restore-test.env	Credenciales libpq montadas read-only en el contenedor (bases principal y de prueba).	No
local-ca.crt	CA raíz de Avast exportada (trabajo en curso) para NODE_EXTRA_CA_CERTS: que n8n confíe en el TLS interceptado por el antivirus.	No (pendiente de verificar)

8. Cómo se relacionan los cuatro flujos

- **WF1 → reacción:** WF1 no depende de WF2: funciona con Prometheus + MantisBT únicamente.
- **WF2 → producción:** WF2 produce el archivo que WF-RESTORE valida de verdad. Sin dumps en backups/, WF-RESTORE no puede probar nada (etapa=seleccionar).
- **WF-RESTORE → verificación:** WF-RESTORE es la garantía honesta del punto 13: confirma que el respaldo de WF2 es restaurable (y cuántos datos contiene), no solo “legible”.
- **WF3 → síntesis:** WF3 consume las mismas métricas que alimentan a WF1 y las resume semanalmente para el equipo: es el “reporte gerencial” del estado de la plataforma.

9. Conclusión

La automatización de n8n está armada con piezas estándar y criterios defensivos: verificación en capas para el backup (existe, pesa, es fresco, es legible), una arena desechable para la prueba de restauración, deduplicación de incidentes por huella, logs legibles como sistema de registro y credenciales fuera del código. WF2 y WF-RESTORE ya demostraron funcionar de punta a punta (backup_exitoso y restore_ok con 199 filas), y WF3 completa el circuito de síntesis semanal asistido por heurísticas. Los pendientes son operativos: desplegar informe-analyzer, importar y reactivar los flujos, resolver el TLS de Avast/SMTP, corregir la clave de reinicios en WF3 y decidir qué hacer con backups/.